

Raden Language Manual

Raden project

Raden is a small stack-based scripting language with a C interpreter, standard libraries, and a native module API.

Contents

1	Introduction	2
2	Quick Start	2
2.1	Install	2
2.2	Build	2
2.3	REPL	2
2.4	Tests	2
3	Values and Syntax	2
3.1	Value Types	2
3.2	Numbers	3
3.3	Strings	3
3.4	Booleans and Null	3
3.5	Lists	3
3.6	Comments	3
4	Operators	4
5	Variables	4
5.1	let	4
5.2	set	4
5.3	const	4
5.4	unlet	5
5.5	enum and reset	5
5.6	Built-in Environment Variables	5
5.7	Identifier Semantics	5
6	Functions	5
6.1	defun	5
6.2	apply	5
6.3	call	6
6.4	pcall	6
6.5	Scope	6
7	Control Flow	6
7.1	if	6
7.2	loop	6
7.3	break, continue, and exit	7

8 Builtins	7
9 Lists and Strings	7
10 Error Handling	8
11 Loading Code	8
11.1 Raden Scripts	8
11.2 Native Libraries	8
12 Standard Library	8
12.1 core	9
12.2 lists	9
12.3 strings	9
12.4 io	9
12.5 files	9
12.6 math	9
12.7 os, unix, path, process, net, json, strconv, flag	10
13 Native Modules	10
13.1 Bundled Native Modules	10
13.2 Native Stack API	10
13.3 Native List API	11
13.4 Native Module Example	11
14 Embedding	11
15 Editor Support	12
16 Test Suite	12
17 Implementation Notes	12

1 Introduction

Raden is built around a value stack. Values are pushed first, then operators and builtins consume values from the stack and push results back. This makes the language postfix:

```
2 3 + print
```

This program pushes 2, pushes 3, applies +, and prints the result. Operand order matters. The expression `i 10 <` means `i < 10`; the expression `10 i <` means `10 < i`.

The interpreter is written in C. The public embedding entry point is:

```
int radn_main(int argc, char **argv);
```

2 Quick Start

2.1 Install

```
curl -fsSL https://raw.githubusercontent.com/abdorayden/rdn/master/install | sh

# or from a cloned checkout
./install
```

By default, libraries install to `/usr/local/share/rdn/libs` and native modules install to `/usr/local/share/rdn/nativelibs`.

2.2 Build

```
git clone https://github.com/abdorayden/rdn.git
cd radn
make
./main script.rdn
```

2.3 REPL

Run `./main` with no script path to start the REPL:

```
$ ./main
raden repl
press Ctrl-D to exit
RDN >> 2 3 + print
```

2.4 Tests

```
bash test/run.sh
```

3 Values and Syntax

3.1 Value Types

Type	Syntax	Example
Integer	123	42

Double	123.456	3.14
String	"..."	"hello"
Boolean	true, false	true
Null	null	null
List	(...)	(1 2 "three")
Identifier	bare word	x, hello

3.2 Numbers

```
10
2.5
```

3.3 Strings

Strings use double quotes. Common escapes include `\n`, `\t`, `\"`, `\\`, and `\e`.

```
"hello"
"line1\nline2"
"quote: \"
```

3.4 Booleans and Null

```
true
false
null
```

3.5 Lists

Lists use parentheses:

```
(1 2 3)
("hello" 4 true)
```

List literals execute their contents. Values left on the temporary list stack become list items. This means conditionals, loops, builtins, and function calls can run inside a list literal:

```
(2 3 1 true if 10 end)

0 i let
(i 10 < loop
  i
  i 1 + i set
  i 10 <
end) print
```

3.6 Comments

```
[* this is a comment *]
42 [* inline comment *] print
```

4 Operators

Operators consume operands from the stack.

Operator	Meaning
+	addition
-	subtraction
*	multiplication
/	division
=	equality
!=	inequality
<	less than
>	greater than
<=	less than or equal
>=	greater than or equal
!	boolean not
&	boolean and, or integer bitwise and
	boolean or, or integer bitwise or
^	integer bitwise xor
<<	integer left shift
>>	integer right shift

```
2 3 + print
10 3 - print
5 3 > print
true false & print
4 1 << print
```

5 Variables

5.1 let

`let` binds a value to a mutable name:

```
10 x let
x print
```

5.2 set

`set` updates an existing binding:

```
x 1 + x set
x print
```

5.3 const

`const` creates an immutable binding. Reassigning a constant is an error:

```
100 limit const
limit print
```

5.4 `unlet`

`unlet` removes a binding:

```
10 x let
x unlet
```

5.5 `enum` and `reset`

```
enum ONE const
enum TWO const
enum THREE const
reset
```

5.6 Built-in Environment Variables

Variable	Value
<code>__host_os</code>	host OS name, such as <code>"linux"</code>
<code>__sharedlib_ext</code>	native library extension, such as <code>".so"</code>
<code>__argv</code>	script path followed by extra CLI arguments
<code>__line_col</code>	source location helper for the called variable

5.7 Identifier Semantics

Identifiers are context-sensitive. Names used before `let`, `set`, `const`, `defun`, or `call` are treated as name operands. Normal variable lookup may clone the underlying value, while list and string variables are often preserved as named targets for mutation-oriented builtins such as `append`, `remove`, `index`, and `len`.

When a script needs a copied list value before mutation, `dup` may be required:

```
next dup current set pop
```

6 Functions

6.1 `defun`

```
double defun
  2 *
end

5 double call print
```

6.2 `apply`

`apply` defines a function-like word that is invoked directly, without `call`:

```
triple apply
  3 *
end

4 triple print
```

6.3 call

`call` invokes a function by name. A string can also name the function:

```
"double" call call
```

6.4 pcall

`pcall` performs a protected call and catches runtime errors. On success the stack receives **result** **true**; on error it receives **"error message"** **false**.

```
risky defun
  null 0 index
end

risky pcall
if
  "success: " print print "\n" print
else
  "error: " print print "\n" print
end
```

6.5 Scope

Functions create a new variable scope. Bindings created inside a function are local unless **set** updates an existing outer binding.

7 Control Flow

7.1 if

`if` consumes one boolean. **else** is optional. Blocks terminate with **end**.

```
true if
  "yes" print
else
  "no" print
end
```

7.2 loop

`loop` consumes a boolean condition at entry. Each iteration must leave the next boolean condition on the stack.

```
0 i let
i 5 < cond let

cond loop
  i print "\n" print
  i 1 + i set
  i 5 < cond set
  cond
end
```

7.3 break, continue, and exit

```
0 i let
true loop
  i 10 > if break end
  i 5 = if i 1 + i set continue end
  i print "\n" print
  i 1 + i set
  true
end
0 exit
```

8 Builtins

Builtin	Stack shape	Description
print	value ->	Print to stdout
pop	value ->	Discard top value
dup	value -> value value	Duplicate top value
swap	a b -> b a	Swap top two values
type	value -> string	Return type name
to_string	value -> string	Convert to string
let	value name ->	Create mutable binding
set	value name ->	Update binding
const	value name ->	Create constant binding
unlet	name ->	Remove binding
enum	-> integer	Push next enum integer
reset	->	Reset enum counter
index	target index -> value	Index list or string
append	target value -> target	Append to list or string
remove	target index -> target	Remove item or character
len	target -> integer	Length of list or string
load	path ->	Execute a Raden source file
loadnative	path ->	Load a native shared library
add_load_path	path ->	Add script search path
add_native_path	path ->	Add native search path
defun	name ... end ->	Define named function
apply	name ... end ->	Define direct-call word
call	name args... -> results...	Invoke a function
pcall	name args... -> result true error false	Protected call
error	string -> error	Raise an interpreter error
if	cond ... end ->	Conditional execution
loop	cond ... next-cond end ->	Loop while true
break	->	Exit loop
continue	->	Continue loop
exit	status ->	Exit process

9 Lists and Strings

```
(10 20 30) 1 index print
(1 2) 3 append print
(1 2 3) 1 remove print
(1 2 3) len print
```

The same sequence builtins work on strings:

```
"hello" 1 index print
"hello" 1 remove print
"hi" len print

"hi" text let
text "!" append
text print
```

10 Error Handling

`pcall` is the primary error-handling mechanism. It invokes a function and catches errors raised during execution.

Situation	Stack after <code>pcall</code>
Success	... <code>function_result true</code>
Error caught	... <code>"error message" false</code>

Errors that happen before the function executes, such as a missing function name, propagate normally.

11 Loading Code

11.1 Raden Scripts

```
"./libs/math.rdn" load
4 sqrt call print
```

Paths are resolved relative to the currently running source file and then through configured search paths.

11.2 Native Libraries

```
"../nativelibs/files.so" loadnative
"./Makefile" readLines call print
```

Use `__sharedlib_ext` for cross-platform native library paths.

```
"../nativelibs/math" __sharedlib_ext to_string swap pop append loadnative
```

12 Standard Library

The standard library is stored in `libs/`. Functions are generally loaded with `load` and called with `call`.

12.1 core

`libs/core.rdn` provides aliases and general helpers:

- Operator aliases: `not`, `add`, `sub`, `mul`, `div`.
- Comparisons: `eq`, `neq`, `lt`, `gt`, `le`, `ge`.
- Boolean and bitwise helpers: `and`, `or`, `xor`, `shl`, `shr`.
- Type predicates: `is_null`, `is_int`, `is_float`, `is_bool`, `is_str`, `is_list`.
- Numeric helpers: `inc`, `dec`, `neg`, `abs`, `even`, `odd`, `min`, `max`, `between`, `clamp`.
- Functional helpers: `pipe`, `tap`, `select`, `when`, `unless`, `times`, `range`, `in`.

12.2 lists

`libs/lists.rdn` provides list helpers: `each`, `foreach`, `map`, `collect`, `filter`, `reduce`, `copy`, `reverse`, `concat`, `first`, `head`, `last`, `tail`, `rest`, `count`, `isEmpty`, `includes`, `any`, `all`, `shift`, `popLast`, `unpack`, and `insert`.

```
(1 2 3) dup collect call print
(1 2 3) 2 > filter call print
(1 2 3) 0 add reduce call print
```

12.3 strings

`libs/strings.rdn` loads the native strings module and provides `concat`, `concatBy`, `contains`, `hasPrefix`, `hasSuffix`, `trimSpace`, `split`, and `replaceAll`.

12.4 io

`libs/io.rdn` provides `readLine`, `readln`, `readAll`, `prompt`, `nl`, `println`, `say`, `printAll`, `printLines`, and `printJoin`.

12.5 files

`libs/files.rdn` provides `readLines`, `writeLines`, `appendLines`, `readText`, `writeText`, `appendText`, `slurp`, and `spit`.

```
"./libs/files.rdn" load
"./data.txt" readLines call lines let
lines each call print
```

12.6 math

`libs/math.rdn` provides `sqrt`, `powf`, and `pow`. It also exposes constants such as `M_E`, `M_PI`, `M_LN2`, `M_SQRT2`, and related C math constants.

12.7 os, unix, path, process, net, json, strconv, flag

Library	Main functions
os	pid, env, sleep, now
unix	pwd, exists, mkdir, rmPath, ls
path	join, basename, dirname, extname, absolute
process	status, output
net	host, encodeUrl, decodeUrl
json	parse, stringify
strconv	atoi, atof, parseBool, itoa, formatBool
flag	resetFlags, addFlag, execute, executeOn, positionals, programName, printHelp

13 Native Modules

Native modules are shared libraries written in C. They export:

```
bool rdn_module_init(RDNModule *module);
```

13.1 Bundled Native Modules

Module	Registered functions
files	readLines, readText, writeLines, appendLines, writeText, appendText
io	readLine, readAllInput
json	parseJson, stringifyJson
math	powerOf, sqrtOf
net	hostName, urlEncode, urlDecode
path	joinPath, baseName, dirName, extName, isAbsolutePath
process	execStatus, execOutput
strconv	atoi, atof, parseBool, itoa, formatBool
strings	strContains, strHasPrefix, strHasSuffix, strTrimSpace, strSplit, strReplaceAll
syscall	pid, getEnv, sleepMs, epochTime
unix	cwd, pathExists, makeDir, removePath, listDir

13.2 Native Stack API

Function	Description
api->stack_size(api)	Number of items on the stack
api->type(api, index)	Type of value at index; -1 is top
api->to_integer(api, index, &out)	Read integer
api->to_number(api, index, &out)	Read double
api->to_boolean(api, index, &out)	Read boolean
api->to_string(api, index)	Read string as const char *
api->pop(api, count)	Pop values
api->push_integer(api, value)	Push integer
api->push_number(api, value)	Push double
api->push_boolean(api, value)	Push boolean
api->push_string(api, value)	Push copied string

<code>api->raise_error(api, message)</code>	Raise an error and return false
--	---------------------------------

13.3 Native List API

Function	Description
<code>api->push_list(api)</code>	Push an empty list
<code>api->list_len(api, index, &len)</code>	Read list length
<code>api->list_append(api, list_idx, value_idx)</code>	Append cloned value
<code>api->list_index(api, list_idx, item_idx)</code>	Push cloned item
<code>api->list_remove(api, list_idx, item_idx)</code>	Remove item

13.4 Native Module Example

```
#include "../include/rdn_native.h"

static bool native_add(RDNApi *api) {
    long left = 0, right = 0;

    if (api->stack_size(api) < 2)
        return api->raise_error(api, "add requires 2 operands");

    if (!api->to_integer(api, -2, &left) ||
        !api->to_integer(api, -1, &right))
        return api->raise_error(api, "add requires integers");

    api->pop(api, 2);
    return api->push_integer(api, left + right);
}

bool rdn_module_init(RDNModule *module) {
    return module->register_function(module, "add", native_add);
}
```

```
gcc -Wall -Wextra -Werror -ggdb -std=c11 -fPIC -shared \
    mymodule.c -I. -o mymodule.so
```

```
"./mymodule.so" loadnative
3 4 add call print
```

14 Embedding

Raden exposes a minimal embedding API through `include/src.h`.

```
#include "../include/src.h"

int main(void) {
    char *argv[] = {
        "host",
        "./script.rdn",
    };
    return rdn_main(2, argv);
}
```

The first argument is the program name and the second is the script path. If `argc < 2`, `rdn_main` starts the REPL. Source-relative `load` and `loadnative` path resolution work in embedded usage.

15 Editor Support

The repository includes Neovim runtime files:

```
cp -r ./nvim/* ~/.config/nvim/
```

This provides file detection for `.rdn`, syntax highlighting, ftplugin defaults, and basic indent rules. A starter Treesitter grammar is available in `treesitter/raden/`.

16 Test Suite

Raden includes regression tests in `test/`. Each test is a `.rdn` file with expected output in `test/expected/`. The runner compares stdout and stderr and verifies the exit code.

```
bash test/run.sh
```

Useful tests include: `test/bugs.rdn`, `test/lists.rdn`, `test/loop.rdn`, `test/functions.rdn`, `test/strings_and_type.rdn`, and `test/rule_110.rdn`.

17 Implementation Notes

When editing the interpreter:

1. Remember that the language is postfix; many apparent bugs are operand-order mistakes.
2. List literals execute code and collect resulting stack values.
3. `type` recognizes function identifiers.
4. Identifier handling is not completely uniform because some values are cloned while list and string names may be preserved for mutation.
5. Validate changes with `make` and `bash test/run.sh`.